Fitness Tracker

DBMS mini-project

Sr No	UID	Name
1	2025800047	Haddi Eba Jaffry
2	2025800048	Lakshya Jain
3	2025800049	Nishad Joshi

1. Case Study Overview

This project designs a MySQL relational database for a fitness tracker. Users log their daily meals and physical exercises, enabling the system to compute daily calorie consumption, calories burnt, and net calorie balance.

1.1 Problem Statement

Individuals aiming to monitor their health or manage weight typically log meals and workouts using informal notes or disconnected spreadsheets. These records are often scattered, inconsistent, tedious to aggregate, and prone to data loss. A structured relational database eliminates data duplication, enforces entry constraints, and seamlessly generates daily and weekly health summaries via queries.

1.2 Objectives

- Store structured user profiles, food catalogues, and exercise catalogues.
- Record daily nutritional intake and physical activities per user.
- Dynamically compute total calories in, calories out, and net balances.
- Maintain chronological logs of weekly body metrics (height and weight) to evaluate progress.
- Guarantee data integrity using primary/foreign keys, check constraints, and maintain a 3NF normalized design.

1.3 Scope

- **In Scope:** User profiles, food & exercise catalogues, daily food/exercise logs, weekly body measurements, and summary queries.
- **Out of Scope:** Authentication login credentials, graphical web/mobile front-ends, wearable API syncing, automated meal recommendations, and payment systems.

1.4 Tools Used

- **MySQL 8.0:** Utilized for schema creation, constraints, and plain SQL queries.
- **ER Modeling & Normalization:** Conceptual design mapped to a relational schema adhering to 3NF standards.

2. Requirement Identification

2.1 Requirements:

- Register users with a unique username, contact details, and date of birth.
- Maintain a food catalogue detailing food name, type, and calories per serving.
- Maintain an exercise catalogue tracking activity name, type, and calories burnt per minute.
- Restrict users to at most one daily tracking record per calendar date.
- Allow users to log multiple food items per day with corresponding serving counts.
- Allow users to log multiple exercises per day with duration parameters in minutes.
- Compute total calorie intake, calories burnt, and net calories per day.
- Store weekly height and weight checkpoints for individual users.
- Calculate mean daily calorie metrics across weekly intervals.

2.2 Data Requirements

- **User Data:** `user_id`, `username`, `user_contact`, `dob`.
- **Catalogue Data:** `Food` (`name`, `food_type`, `calories_per_serving`), `Burn` (`burn_name`, `burn_type`, `calories_per_min`).
- **Log Data:** `record_date`, `servings`, `duration_min`.
- **Body Metrics:** `height_cm`, `weight_kg`, `week_start_date`.

2.3 Business Rules

- Usernames must be unique and non-null.
- Future dates are disallowed for daily and profile records.
- Duplicate entries of the same food or exercise on a given day are prevented by updating quantities instead of spawning redundant rows.
- Quantities (servings, durations) must be strictly positive (> 0); calorie values cannot be negative.

- Historical food and exercise catalogue entries cannot be deleted if referenced by existing user logs (`ON DELETE RESTRICT`). Deleting a user cascades and cleans up dependent logs.

3. Entities, Attributes, and Dependencies

3.1 Entity Description Table

Entity Name	Entity Type	Purpose	Attributes
Users	Strong	People tracking fitness	`user_id`, `username`, `user_contact`, `dob`
Food	Strong	Catalogue of available foods	`food_id`, `name`, `food_type`, `calories_per_serving`
Burn	Strong	Catalogue of physical exercises	`burn_id`, `burn_name`, `burn_type`, `calories_per_min`
Daily_Record	Dependent	One tracked day per user	`daily_id`, `user_id`, `record_date`
Daily_Food	Associative	Foods consumed on a specific day	`daily_id`, `food_id`, `servings`
Daily_Burn	Associative	Exercises performed on a specific day	`daily_id`, `burn_id`, `duration_min`
Weekly_Record	Dependent	Weekly anthropometric metrics	`week_id`, `user_id`, `week_start_date`, `height_cm`, `weight_kg`

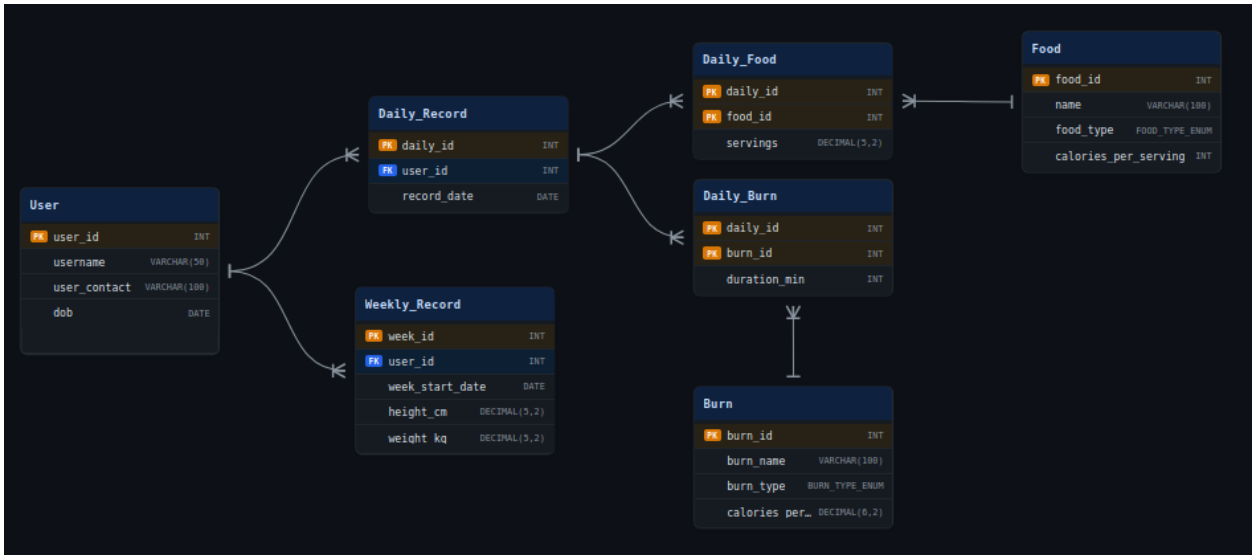
3.2 Relational Schema & Primary/Foreign Keys

- **`Users` (`user_id` [PK], `username`, `user_contact`, `dob`)**
- **`Food` (`food_id` [PK], `name`, `food_type`, `calories_per_serving`)**
- **`Burn` (`burn_id` [PK], `burn_name`, `burn_type`, `calories_per_min`)**
- **`Daily_Record` (`daily_id` [PK], `user_id` [FK references Users], `record_date`)**
- **`Daily_Food` (`daily_id` [FK references Daily_Record], `food_id` [FK references Food], `servings`, [PK: daily_id, food_id])**
- **`Daily_Burn` (`daily_id` [FK references Daily_Record], `burn_id` [FK references Burn], `duration_min`, [PK: daily_id, burn_id])**
- **`Weekly_Record` (`week_id` [PK], `user_id` [FK references Users], `week_start_date`, `height_cm`, `weight_kg`)**

4. Normalization (3NF Compliance)

- **1NF (First Normal Form):** All columns contain atomic values, and repeating groups are eliminated through associative tables (`Daily_Food`, `Daily_Burn`).
- **2NF (Second Normal Form):** The database satisfies 1NF and all non-key attributes are fully functionally dependent on the primary key.
- **3NF (Third Normal Form):** No transitive dependencies exist. Derived fields (such as total daily calories, net calories, and age) are computed dynamically via queries rather than being redundantly stored, preventing update anomalies.

5. ER Diagram:


User
user_id INT
username VARCHAR(50)
user_contact VARCHAR(100)
dob DATE
Daily_Record
daily_id INT
user_id INT
record_date DATE
Weekly_Record
week_id INT
user_id INT
week_start_date DATE
height_cm DECIMAL(5,2)
weight_kg DECIMAL(5,2)
Daily_Food
daily_id INT
food_id INT
servings DECIMAL(5,2)
Daily_Burn
daily_id INT
burn_id INT
duration_min INT
Food
food_id INT
name VARCHAR(100)
food_type FOOD_TYPE_ENUM
calories_per_serving INT
Burn
burn_id INT
burn_name VARCHAR(100)
burn_type BURN_TYPE_ENUM
calories per… DECIMAL(6,2)